



Módulo 3

Personalizando la Apariencia de Aplicaciones WPF



1- Personalizar Apariencia

Composición

Un requisito habitual para personalizar la apariencia de un control es crear controles compuestos, o sea que incluyan más de un control. La mayoría de los controles WPF son contenedores de otros controles. Puede usar la composición para incrustar los elementos XAML dentro de otros. En varios de los ejemplos mostrados en el capítulo anterior usamos esta técnica para la creación de los controles.

```
<Button Name="btn5" Canvas.Left="64" Canvas.Top="17" Width="129" Height="33"
Click="btnAceptar_Click" >
  <StackPanel Orientation="Horizontal" Width="Auto">
    <Image Source="ms.jpg" Width="20" Height="20"></Image>
    <TextBlock Text="Aceptar" TextAlignment="Right" VerticalAlignment="Center"
Width="69"></TextBlock>
  </StackPanel>
</Button>
```

Estilos

Los estilos y plantillas permiten el mantenimiento y el uso compartido de la apariencia tanto dentro de una aplicación como entre las diversas aplicaciones. Hacen referencia a un conjunto de características que permiten a los diseñadores de aplicaciones, documentos o interfaces de usuario crear aplicaciones visualmente atractivas y estandarizar la apariencia del producto.

Otra característica del modelo de estilos de WPF es la separación de la presentación y la lógica. Esto significa que los diseñadores pueden trabajar en la apariencia de una aplicación utilizando solamente XAML, al mismo tiempo que los desarrolladores trabajan en la lógica de programación utilizando C# o Visual Basic.

La clase **Style** permite generar los estilos en aplicaciones WPF. Puede establecer una clase *Style* en cualquier elemento que derive de *FrameworkElement* o *FrameworkContentElement*. Un estilo se suele declarar normalmente como un recurso de la sección de *Resources*. Puesto que los estilos son recursos, siguen las mismas reglas de ámbito que se aplican a todos los recursos; por tanto, cuando se declara un estilo, afecta a la parte donde se puede aplicar. Si, por ejemplo, se declara el estilo en el elemento raíz del archivo XAML de definición de la aplicación, el estilo se puede usar en cualquier parte de la aplicación. Si se crea una aplicación de navegación y se declara el estilo en uno de sus archivos XAML, el estilo sólo se puede usar en ese archivo XAML.

La declaración de estilo está formada por un objeto **Style** que contiene una colección de uno o más objetos **Setter**. Cada objeto *Setter* consta de una propiedad **Property** y una propiedad **Value**. La propiedad es el nombre de la propiedad del elemento al que se aplica el estilo. Una vez declarado el estilo como recurso, se puede hacer referencia a él como en el caso de cualquier otro recurso. Si hubiera muchos estilos con la misma propiedad *Property* es establece el valor del declarado en último lugar.

Si se establece la propiedad **TargetType** en un control sin asignar al estilo un atributo **x:Key**, el estilo se aplicará a todos los elementos de ese tipo. Esto significa que si se establece explícitamente el atributo **x:Key** en un valor, el objeto *Style* no se aplicará automáticamente a todos los elementos de ese tipo. Si el estilo está en la sección de recursos y no se establece la propiedad *TargetType* del estilo, se deberá proporcionar un atributo **x:Key**.

En el ejemplo siguiente se define un estilo que se aplicará a todas las instancias del elemento *Button*. Como se ve en el ejemplo los botones toman automáticamente el estilo sin necesidad de asociarlo por código.

```
<Window.Resources>
```

```
<Style TargetType="{x:Type Button}">
  <Setter Property="FontFamily" Value="Times New Roman" />
  <Setter Property="FontSize" Value="30" />
  <Setter Property="FontWeight" Value="Bold" />
  <Setter Property="Background" Value="#FFCA5132" />
</Style>
</Window.Resources>
<Button Height="54" Name="button1" Width="134">Button</Button>
```



En el siguiente ejemplo se muestra una declaración de estilo que afectará a la propiedad Background de una clase Control. La asignación del estilo a un control se hace en forma explícita.

```
<Window.Resources>
  <Style x:Key="Style1">
    <Setter Property="Control.Background" Value="Yellow"/>
  </Style>
</Window.Resources>
<StackPanel>
  <Label Content="Label Amarillo" Style="{StaticResource Style1}"
    Width="100"/>
  <Button Name="button1" Style="{StaticResource Style1}" Height="20"
    Width="50">Button</Button>
</StackPanel>
```



En el siguiente ejemplo el estilo se aplica al control StackPanel y sus controles contenidos y no a toda la ventana.

```
<StackPanel>
  <StackPanel.Resources>
    <Style TargetType="Button">
      <Setter Property="Background">
        <Setter.Value>
          <LinearGradientBrush>
            <GradientStop Color="#DDDDDD" Offset="0" />
            <GradientStop Color="#88FF88" Offset=".6" />
            <GradientStop Color="#EEEEEE" Offset="1" />
          </LinearGradientBrush>
        </Setter.Value>
      </Setter>
    </Style>
  </StackPanel.Resources>
```

```
<Button Height="50" Width="50">
  <StackPanel>
    <TextBlock>Play</TextBlock>
    <Polygon Points="0,0 0,26 17,13"
      Fill="Black" />
  </StackPanel>
</Button>

<Button Height="50" Width="50">
  <StackPanel>
    <TextBlock>Play</TextBlock>
    <Polygon Points="0,0 0,26 17,13"
      Fill="Black" />
  </StackPanel>
</Button>
</StackPanel>
```



Nota: Muchos controles de WPF constan de una combinación de otros controles de WPF, por lo que la creación de un estilo que se aplique a todos los controles de un tipo puede tener un gran impacto. Por ejemplo, si se crea un estilo cuyo destino son los controles *TextBlock* de un control *Canvas*, el estilo se aplica a todos los controles *TextBlock* del lienzo, aunque *TextBlock* forme parte de otro control, como *ListBox*.

Plantillas

Los estilos están limitados a la configuración de las propiedades predeterminadas en elementos XAML. Para una libertad completa de una apariencia del control, necesita usar plantillas. Para ello, debe crear un estilo y especificar la propiedad de plantilla. El valor de la propiedad de plantilla pasa a ser un elemento **ControlTemplate** que especifica cómo redactar el control.

La mayoría de controles tienen una apariencia y un comportamiento. Considere un botón: la apariencia es el área elevada que se puede presionar y el comportamiento es el evento Click que se provoca en respuesta a un clic. En ocasiones, puede haber un control que proporcione el comportamiento necesario, pero no la apariencia necesaria. Hasta ahora, hemos mostrado que puede utilizar establecedores de estilo para establecer valores de propiedad que afecten a la apariencia del control. Sin embargo, para cambiar la estructura de un control o establecer valores de propiedad en los componentes de un control, debe utilizar un objeto *ControlTemplate*.

En WPF, el objeto *ControlTemplate* de un control define su apariencia. Puede cambiar la estructura y la apariencia de un control definiendo un nuevo objeto *ControlTemplate* para el control. En muchos casos, esto ofrece suficiente flexibilidad como para no tener que escribir controles personalizados.

En el siguiente ejemplo se ha especificado un botón en forma de círculo con el icono de reproducción en el centro, colocando el icono de reproducción sobre un elemento *ellipse*. El nuevo botón de plantilla aparece junto a un botón normal.

```
<StackPanel>
  <StackPanel.Resources>
    <Style TargetType="{x:Type Button}" x:Key="PlayButton" >
      <Setter Property="Template">
        <Setter.Value>
          <ControlTemplate>
            <Grid>
              <Ellipse Width="{TemplateBinding Width}"
                Height="{TemplateBinding Height}" Stroke="DarkGray"
                VerticalAlignment="Top"
                HorizontalAlignment="Left" Fill="LightGray" />
              <Polygon Points="18,12 18,38 35,25" Fill="Black" />
            </Grid>
          </ControlTemplate>
        </Setter.Value>
      </Setter>
    </Style>
  </StackPanel.Resources>
  <Button Height="50" Width="100">Botón Normal</Button>
  <Button Height="50" Width="50" Style="{StaticResource PlayButton}" />
</StackPanel>
```



Otro ejemplo de cómo aplicar plantillas. Tenga en cuenta que la plantilla se aplica siempre a través de un estilo. Este ejemplo usa una plantilla que a través de un control Border redondea la forma de los botones.

```
<Window.Resources>
  <ControlTemplate x:Key="PlantBotonAzul" TargetType="{x:Type Button}">
    <Border BorderBrush="Navy" BorderThickness="1" CornerRadius="5"
      Background="CornflowerBlue">
      <Grid>
        <Grid.RowDefinitions>
          <RowDefinition />
        </Grid.RowDefinitions>
        <ContentPresenter Grid.Row="1" HorizontalAlignment="Center"
          VerticalAlignment="Center" />
      </Grid>
    </Border>
  </ControlTemplate>
  <Style x:Key="EstiloBotonAzul" TargetType="{x:Type Button}">
    <Setter Property="Template" Value="{StaticResource PlantBotonAzul}" />
  </Style>
</Window.Resources>
<Button Height="40" Name="button1" Width="120" Content="Aceptar"
  Style="{StaticResource EstiloBotonAzul}" />
```

Aceptar

Desencadenadores (Triggers)

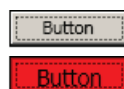
La clase **Trigger** representa un desencadenador que aplica valores de propiedad o realiza acciones si se cumplen determinadas condiciones. WPF define propiedades que se corresponden con acciones del usuario final, como la propiedad *IsMouseOver* cuyo valor se establece en true cuando el usuario desplaza el puntero sobre un objeto, o bien, la correspondiente propiedad *IsMouseOver* de un objeto. La representación de las acciones del usuario final en los valores de propiedad junto con el elemento Trigger permite a los estilos de WPF cambiar en el marcado los valores de las propiedades según esas acciones del usuario final.

Las propiedades cambiadas por los desencadenadores se restablecen automáticamente en su valor anterior si ya no se cumple la condición de activación. Los desencadenadores se optimizan para los estados transitorios de los que se espera que cambien y vuelvan a su estado original, como *IsPressed* en el control *Button* e *IsSelected* en el control *ListBoxItem*. La propiedad *Property* de interés debe ser una propiedad de dependencia.

Observe que debe especificar las propiedades *Property* y *Value* en un objeto *Trigger* para que el desencadenador tenga sentido. La propiedad *Setters* de un objeto *Trigger* sólo puede estar formada por objetos *Setter*.

En el siguiente ejemplo se muestra el uso de desencadenadores de manera que cada vez que se hace clic sobre un botón este cambia el tamaño de la letra de su contenido a 14 y cada vez que se pasa el Mouse por encima del mismo el color del fondo se vuelve rojo.

```
<Window.Resources>
  <Style x:Key="TrCambioColor" TargetType="{x:Type Button}">
    <Style.Triggers>
      <Trigger Property="IsPressed" Value="true">
        <Setter Property="FontSize" Value="14"/>
      </Trigger>
      <Trigger Property="IsMouseOver" Value="true">
        <Setter Property="Background" Value="red"/>
      </Trigger>
    </Style.Triggers>
  </Style>
</Window.Resources>
<StackPanel >
  <Button Height="23" Name="button1" Width="75" Style="{StaticResource
    TrCambioColor}">Button</Button>
</StackPanel>
```



Otro tipo de desencadenador es **EventTrigger**, que inicia un conjunto de acciones en función de la aparición de un evento. Los objetos *EventTrigger* inician un conjunto de propiedades **Actions** cuando se produce un evento enrutado especificado. Por ejemplo, quizá desee usar *EventTrigger* para iniciar un conjunto de animaciones cuando el puntero del mouse esté sobre



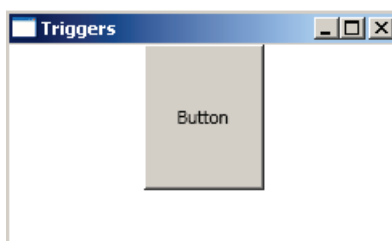
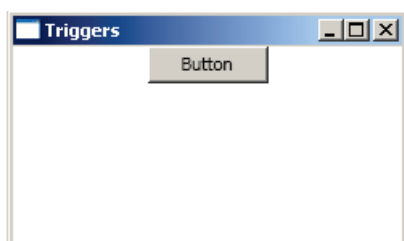
cierto control de la interfaz de usuario (UI). A diferencia de *Trigger*, *EventTrigger* no tiene ningún concepto de finalización de estado, por lo que no se deshará la acción una vez que la condición que provocó el evento deje de ser verdadera.

Tenga en cuenta que al usar *EventTrigger*, debe elegir eventos que no interfieran con el comportamiento inherente del control. Los controles como *Button* o *TextBox* realizan acciones concretas en los eventos de usuario como clics del mouse y eventos de teclado. Por ejemplo, si está creando un botón e intenta establecer el evento *MouseDown* como la propiedad *RoutedEvent* de *EventTrigger*, *EventTrigger* nunca se aplica porque el botón administra primero el evento. En su lugar, puede usar el evento *PreviewMouseDown* o un evento diferente.

En el siguiente ejemplo, los objetos *EventTrigger* especifican que, cuando el puntero del mouse entre en el elemento *Button*, la propiedad *Height* se establezca en 90 durante un período de 0.2 segundos. Cuando el mouse sale del elemento, la propiedad se establece de nuevo en el valor original durante un período de 1 segundo. Note que se escriben ambos eventos de manera que la acción se detenga al cambiar el estado.

```
<Window.Resources>
  <Style x:Key="TrCambioColor" TargetType="{x:Type Button}">
    <Style.Triggers>
      <EventTrigger RoutedEvent="Mouse.MouseEnter">
        <EventTrigger.Actions>
          <BeginStoryboard>
            <Storyboard>
              <DoubleAnimation Duration="0:0:0.2"
                Storyboard.TargetProperty="Height" To="90" />
            </Storyboard>
          </BeginStoryboard>
        </EventTrigger.Actions>
      </EventTrigger>
      <EventTrigger RoutedEvent="Mouse.MouseLeave">
        <EventTrigger.Actions>
          <BeginStoryboard>
            <Storyboard>
              <DoubleAnimation Duration="0:0:1"
                Storyboard.TargetProperty="Height" />
            </Storyboard>
          </BeginStoryboard>
        </EventTrigger.Actions>
      </EventTrigger>
    </Style.Triggers>
  </Style>
</Window.Resources>
<StackPanel >
  <Button Height="23" Name="button1" Width="75" Style="{StaticResource
    TrCambioColor}">Button</Button>
</StackPanel>
```





Además de Trigger y EventTrigger, hay otros tipos de desencadenadores. **MultiTrigger** permite establecer valores de propiedad en función de varias condiciones. **DataTrigger** y **MultiDataTrigger** se utilizan cuando la propiedad de la condición está enlazada a datos.

2- Crear Controles de Usuario

Controles

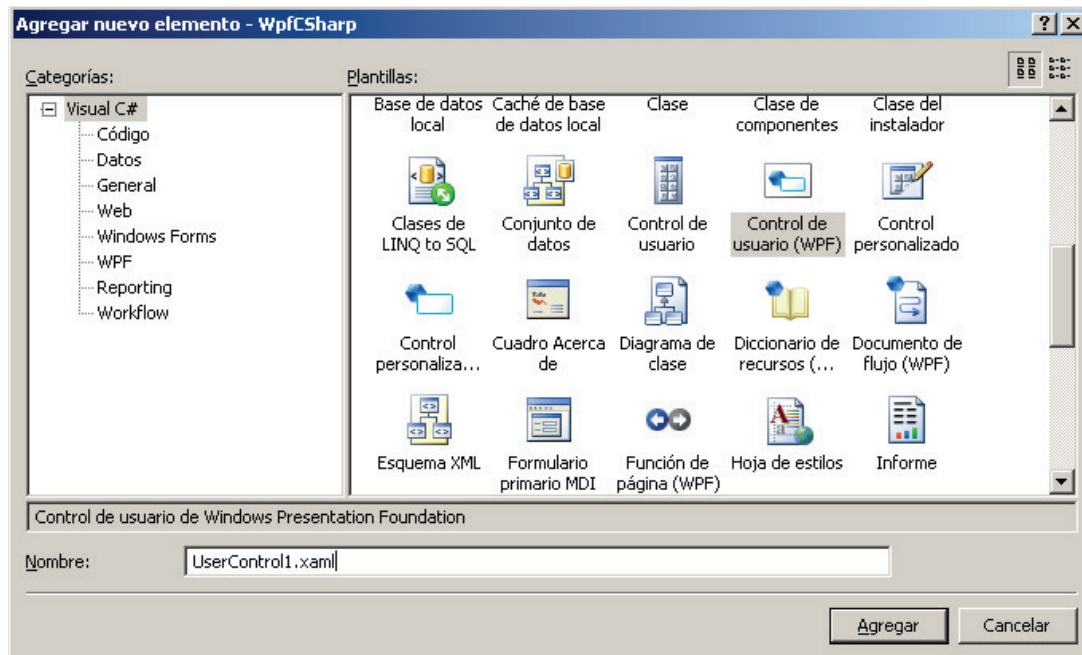
Es imposible suministrar todos los controles que se podrían necesitar. Por este motivo existe la creación de controles. En Windows Presentation Foundation, la composición, el estilo y las plantillas habilitan la personalización de los controles existentes hasta un nivel sin precedentes en tecnologías anteriores, pero la creación de controles permite armar controles una única vez y re usarlos en distintos formularios o proyectos.

El primer paso que debe dar antes de escribir su propio control es decidir qué método va a usar para crearlo. Existen dos maneras principales de crear controles en Windows Presentation Foundation: los controles de usuario y los controles personalizados. Ambos enfoques ofrecen ventajas.

- Controles de usuario: Es un fragmento de una ventana o página. Permite incluir varios controles y su funcionalidad como una forma de encapsulamiento visual. Se los llama también controles compuestos.
- Controles personalizados: Generalmente heredan de un solo control y nos permiten sumar funcionalidad o apariencia al mismo. Su propósito es mejorar los controles existentes.

Controles de Usuario

Lo primero que debe hacer al crear un control de usuario es agregar un elemento nuevo a su proyecto. Si hace clic al proyecto con el botón secundario y luego a Agregar, no elija la opción de Control de Usuario del menú contextual. Lamentablemente, esta función intentará crear un control de usuario nuevo de Windows Forms. En su lugar, escoja la opción Agregar nuevo elemento. En el diálogo Agregar nuevo elemento, escoja el elemento Control de Usuario (WPF).



Al crear el control de usuario nuevo se crea un archivo XAML y un archivo de código de seguridad. El archivo XAML es semejante al archivo principal, creado con proyectos nuevos de Windows Presentation Foundation; la diferencia es que el elemento raíz del archivo XAML nuevo es un elemento **UserControl**. Dentro del elemento *UserControl* debe crear el contenido que configurará su control.

En el siguiente ejemplo se crea un UserControl para armar un botón redondo con el símbolo de play.

```
<UserControl x:Class="WpfCSharp.UserControlWPF"
xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
Height="55" Width="55">
  <Grid>
    <Ellipse Width="50" Height="50" Stroke="DarkGray"
      VerticalAlignment="Top" HorizontalAlignment="Left"
      Name="BotonNegro" Fill="LightGray" />
    <Polygon Name="IconoPlay" Points="18,12 18,38 35,25" Fill="Black" />
  </Grid>
</UserControl>
```

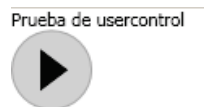
A continuación se muestra el código para usar el control en una ventana. Antes de poder usar un control de usuario en una ventana debe referenciar la ventana al espacio de nombres y darle un alias a dicha referencia. Eso se hace agregando la línea

```
xmlns:UC="clr-namespace:WpfCSharp"
```

donde UC es el nombre que se va a dar al namespace del control. Si el control estuviera en otro ensamblado la línea a ingresar es

```
xmlns:cust="clr-namespace:CustomWPF;assembly=CustomWPF"
```

```
<Window x:Class="WpfCSharp.Window2"
xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
xmlns:UC="clr-namespace:WpfCSharp"
Title="Window2" Height="300" Width="500" >
    <StackPanel>
        <TextBlock>Prueba de usercontrol</TextBlock>
        <UC:UserControlWPF HorizontalAlignment="left" />
    </StackPanel>
</Window>
```



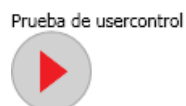
Personalizando Controles de Usuario

Cuando se crean controles muchas veces es necesario generar código no solo para poder interactuar con ellos, sino para encapsular funcionalidad. Al crear los controles descubrirá que es necesario implementar las propiedades, métodos y/o eventos para administrar tanto la apariencia del control como su comportamiento en el tiempo de ejecución. Por ejemplo, sería importante en el control creado en el ejemplo anterior obtener y establecer el color del icono. Para ello, se puede crear una propiedad como se muestra a continuación.

```
Brush _ColorIcono = Brushes.Black;
public Brush ColorIcono
{
    get { return _ColorIcono; }
    set
    {
        _ColorIcono = value;
        IconoPlay.Fill = _ColorIcono;
    }
}
```

Y en la ventana que usa a este control se asigna el valor de la propiedad como cualquiera de las otras propiedades del control.

```
<StackPanel>
    <TextBlock>Prueba de usercontrol</TextBlock>
    <UC:UserControlWPF HorizontalAlignment="left" ColorIcono="Red" />
</StackPanel>
```





También podríamos crear un Control de Usuario WPF y cambiar las referencias a la clase *UserControl* por el control con el cual se desea trabajar, de manera de personalizar solamente un control.

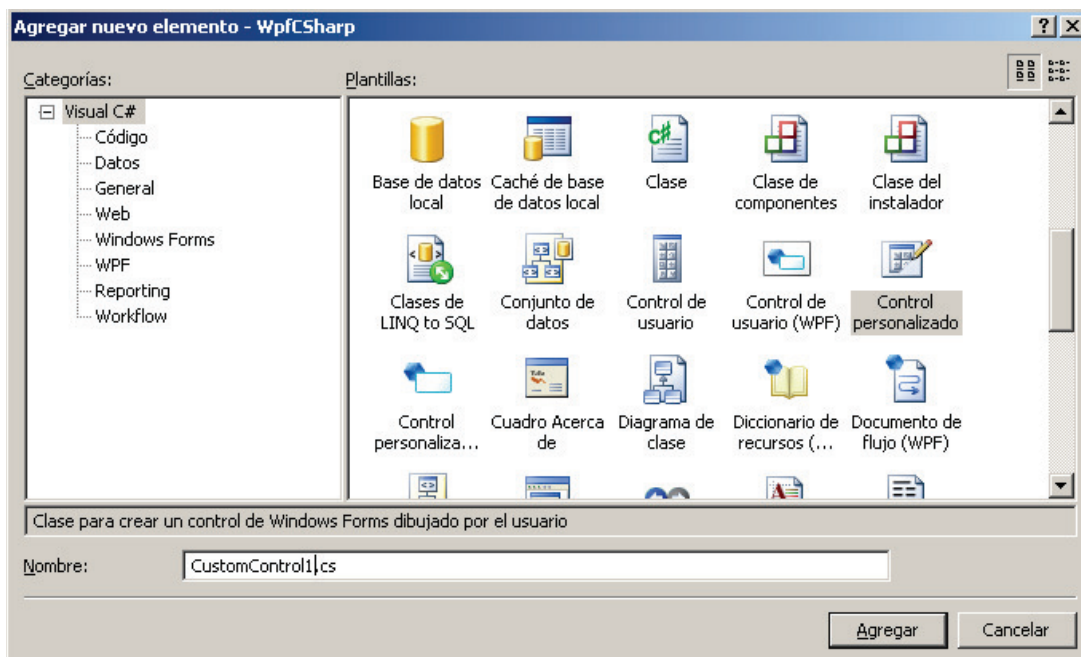
```
<Button x:Class="Pruebas.controlPrueba"
xmlns="http://schemas.microsoft.com/winfx/2006/xaml/presentation"
xmlns:x="http://schemas.microsoft.com/winfx/2006/xaml"
Height="23" Width="100">
```

```
</Button>
```

```
public partial class controlPrueba : Button
{
    public controlPrueba()
    {
        InitializeComponent();
    }
}
```

Controles Personalizados

Si deseamos agregar a la aplicación un control personalizado lo hacemos desde Agregar Elementos, Control Personalizado.



Una vez creado veremos la diferencia con respecto a los controles de usuario. Estos controles no tienen código XAML, por lo tanto necesitamos crear los controles que vayamos a utilizar mediante código subyacente lo cual genera un desarrollo mucho más complejo.